PYTHON CHEAT SHEET

Sumário

1. **Conjuntos (set)**
    - Estrutura de dados para armazenar elementos **únicos** e **não ordenados**.
    - Útil para eliminar duplicatas e realizar operações matemáticas de conjuntos.
2. **Dicionários (dict)**
    - Estrutura de dados que armazena **pares chave-valor**.
    - As chaves são **únicas** e geralmente imutáveis (strings, números, tuplas).
    - Permite acesso rápido aos valores através das chaves.
3. **Deque (collections.deque)**
    - Estrutura de dados da biblioteca collections que representa uma **fila dupla** (Double-Ended Queue).
    - Permite inserção e remoção eficiente de elementos tanto no início quanto no final.
4. **Heap (heapq)**
    - Estrutura de dados baseada em árvore binária, onde o menor elemento sempre está na raiz (min-heap).
    - Útil para implementar filas de prioridade e obter rapidamente o menor elemento.
    - O módulo heapq fornece funções para manipular listas como heaps.
5. **Grafos**
    - Estrutura de dados composta por **vértices** (nós) e **arestas** (ligações entre os nós).
    - Pode ser **direcionado** (arestas com direção) ou **não direcionado**.
6. **Linear Scan**
    - Linear scan (varredura linear) é percorrer sequencialmente uma coleção para buscar, filtrar ou processar elementos.
    - Em Python, muitas tarefas comuns de linear scan podem ser feitas com funções e métodos já disponíveis na linguagem.
7. **Algoritmos Gulosos**
    - Algoritmo guloso (greedy) toma decisões locais ótimas em cada etapa, buscando uma solução global ótima.
    - Em Python, muitos problemas gulosos podem ser resolvidos usando funções e métodos nativos, sem bibliotecas externas.
8. **Programação Dinâmica**

- Programação dinâmica (Dynamic Programming - DP) é uma técnica para resolver problemas complexos dividindo-os em subproblemas menores e reutilizando os resultados já calculados (memoização ou tabulação).
- É útil para problemas de otimização e contagem, como cálculo de Fibonacci, mochila, caminhos mínimos, etc.

9. **Busca Binária**

    - A busca binária é um algoritmo eficiente para encontrar um elemento em uma coleção **ordenada**.
    - Divide repetidamente a coleção ao meio, descartando a metade onde o elemento não pode estar.
    - Complexidade: **O(log n)**.

10. **Permutação e Contagem de Ciclos**

    - Permutação:
        - Permutação é uma reorganização dos elementos de uma sequência em todas as ordens possíveis.
        - Útil para resolver problemas de combinação, ordenação e análise de ciclos.
    - Contagem de Ciclos:
        - Contagem de ciclos é usada para identificar ciclos em permutações ou grafos.
        - Um ciclo é uma sequência de elementos que retorna ao ponto inicial após uma série de trocas.

11. **Algoritmo de Kruskal**

    - O algoritmo de Kruskal é usado para encontrar a **Árvore Geradora Mínima (MST)** de um grafo.
    - Ele funciona escolhendo as arestas de menor peso, garantindo que não formem ciclos, até conectar todos os vértices.

12. **Algoritmo de Dijkstra**

    - O algoritmo de Dijkstra é usado para encontrar o **caminho mais curto** de um vértice de origem para todos os outros vértices em um grafo ponderado.
    - Funciona apenas com **pesos não negativos**.

13. **Bitmask e Busca Exaustiva**

    - **Bitmask** é uma técnica que utiliza operações bit a bit para representar subconjuntos ou estados de forma compacta.
    - É útil para resolver problemas de **busca exaustiva** em que precisamos explorar todas as combinações possíveis de elementos.

14. **Problemas Clássicos**

    - Problema do Caixeiro Viajante (Traveling Salesman Problem - TSP)
        - Dado um conjunto de cidades e as distâncias entre elas, o objetivo é encontrar o caminho mais curto que visita todas as cidades exatamente uma vez e retorna à cidade inicial.

        - É um problema de otimização combinatória.
    - Problema da Torre de Hanói
        - Dado 3 pinos e n discos de tamanhos diferentes, o objetivo é mover todos os discos do pino inicial para o pino final, seguindo estas regras:
        1. Apenas um disco pode ser movido por vez.
        2. Um disco maior nunca pode ser colocado sobre um disco menor.

---

1 Conjuntos (set)

Criando um set

```
# Set vazio
s = set()

# Set com elementos
s = {1, 2, 3}
```

Adicionando e removendo elementos

```
s = {1, 2, 3}
s.add(4)        # Adiciona 4
s.remove(2)     # Remove 2 (erro se não existir)
s.discard(5)    # Remove 5 (NÃO dá erro se não existir)
s.pop()         # Remove e retorna um elemento aleatório
s.clear()       # Remove todos os elementos
```

Operações de conjuntos

```
a = {1, 2, 3}
b = {3, 4, 5}

a | b      # União: {1, 2, 3, 4, 5}
a & b      # Interseção: {3}
a - b      # Diferença: {1, 2}
a ^ b      # Diferença simétrica: {1, 2, 4, 5}
```

Métodos úteis para conjuntos

```
a = {1, 2, 3}
b = {1, 2}

a.issubset(b)      # False (a é subconjunto de b?)
b.issubset(a)      # True
```

```
a.issuperset(b)    # True (a é superconjunto de b?)
a.isdisjoint(b)    # False (não têm elementos em comum?)
len(a)             # 3 (tamanho do set)
```

Iterando sobre um set

```
for elemento in s:
    print(elemento)
```

Observações sobre conjuntos

- Sets **não suportam indexação** (não use `s[0]`).
- Elementos precisam ser **imutáveis** (ex: números, strings, tuplas).

Conversão

```
lista = [1, 2, 2, 3]
s = set(lista)     # {1, 2, 3}
```

Dica: Use sets para eliminar duplicatas rapidamente!

---

2 Dicionários (dict)

Criando um dicionário

```
# Dicionário vazio
d = {}

# Dicionário com elementos
d = {'nome': 'Ana', 'idade': 25}
```

Acessando e modificando valores

```
d = {'nome': 'Ana', 'idade': 25}

# Acessar valor
print(d['nome'])        # Ana

# Modificar valor
d['idade'] = 26

# Adicionar novo par chave-valor
d['cidade'] = 'São Paulo'
```

Métodos úteis para dicionários

```
d = {'a': 1, 'b': 2, 'c': 3}

d.keys()        # dict_keys(['a', 'b', 'c'])
d.values()      # dict_values([1, 2, 3])
d.items()       # dict_items([('a', 1), ('b', 2), ('c', 3)])
d.get('a')      # 1 (retorna None se não existir)
d.pop('b')      # Remove e retorna o valor da chave 'b'
d.popitem()     # Remove e retorna o último par inserido
d.clear()       # Remove todos os itens
```

Iterando sobre um dicionário

```
d = {'a': 1, 'b': 2}

for chave in d:
    print(chave, d[chave])

for chave, valor in d.items():
    print(chave, valor)
```

Verificando existência de chave

```
d = {'x': 10}

'x' in d        # True
'y' in d        # False
```

Observações sobre dicionários

- Chaves devem ser **imutáveis** (ex: string, número, tupla).
- Valores podem ser de qualquer tipo.
- Dicionários **não garantem ordem** em versões < 3.7. A partir do Python 3.7, mantêm ordem de inserção.

Compreensão de dicionários

```
quadrados = {x: x**2 for x in range(5)}  # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Dica: Use dicionários para mapear relações entre dados!

---

3 Deque (collections.deque)

Importando e criando um deque

```
from collections import deque

# Deque vazio
d = deque()

# Deque com elementos
d = deque([1, 2, 3])
```

Adicionando elementos

```
d.append(4)        # Adiciona 4 ao final
d.appendleft(0)    # Adiciona 0 ao início
```

Removendo elementos

```
d.pop()            # Remove e retorna o último elemento
d.popleft()        # Remove e retorna o primeiro elemento
```

Acessando elementos

```
print(d[0])        # Acessa o primeiro elemento
print(d[-1])       # Acessa o último elemento
```

Métodos úteis para deques

```
d.extend([5, 6])         # Adiciona múltiplos elementos ao final
d.extendleft([-2, -1])   # Adiciona múltiplos elementos ao início (ordem
invertida)
d.rotate(1)              # Rotaciona todos os elementos para a direita
d.rotate(-1)             # Rotaciona todos os elementos para a esquerda
d.clear()                # Remove todos os elementos
len(d)                   # Tamanho do deque
```

Exemplo de uso como fila e pilha

```
# Fila (FIFO)
fila = deque()
fila.append('a')
fila.append('b')
fila.popleft()   # Remove 'a'

# Pilha (LIFO)
pilha = deque()
pilha.append('x')
pilha.append('y')
pilha.pop()      # Remove 'y'
```

Dica: Use deque quando precisar de inserções/remoções rápidas nas duas extremidades!

---

4 Heap (heapq)

Importando e criando um heap

```
import heapq

# Lista comum
nums = [5, 1, 8, 3]

# Transformar em heap (min-heap)
heapq.heapify(nums)  # nums agora é [1, 3, 8, 5]
```

Inserindo elementos

```
heapq.heappush(nums, 2)  # Adiciona 2 mantendo a propriedade do heap
```

Removendo o menor elemento

```
menor = heapq.heappop(nums)  # Remove e retorna o menor elemento
```

Acessando o menor elemento (sem remover)

```
menor = nums[0]
```

Substituindo o menor elemento

```
heapq.heapreplace(nums, 4)  # Remove o menor e insere 4 (mais eficiente
que pop + push)
```

Obtendo os menores/maiores elementos

```
import heapq

nums = [7, 2, 5, 3, 9]
heapq.heapify(nums)

menores = heapq.nsmallest(3, nums)  # [2, 3, 5]
maiores = heapq.nlargest(2, nums)   # [9, 7]
```

Usando heap como max-heap

- Por padrão, `heapq` é min-heap. Para simular max-heap, use valores negativos:

```
nums = [5, 1, 8, 3]
max_heap = [-n for n in nums]
heapq.heapify(max_heap)

maior = -heapq.heappop(max_heap)  # Retorna o maior elemento
```

Dica: Use `heapq` para problemas que exigem acesso eficiente ao menor (ou maior) elemento de uma coleção!

---

5 Grafos

Representação de grafos

1. Usando dicionário de listas de adjacência

```
# Grafo não direcionado
grafo = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
```

2. Usando lista de arestas

```
arestas = [
    ('A', 'B'),
    ('A', 'C'),
    ('B', 'D'),
    ('C', 'D')
]
```

Adicionando vértices e arestas

```
grafo = {}

# Adiciona vértice
grafo['A'] = []

# Adiciona aresta
grafo['A'].append('B')
grafo['B'] = ['A']
```

Percorrendo um grafo (BFS e DFS)

Busca em largura (BFS)

A busca em largura (Breadth-First Search) explora o grafo visitando todos os vértices vizinhos de um nó antes de avançar para os vizinhos dos vizinhos. Utiliza uma fila (queue) para controlar a ordem de visitação. É útil para encontrar o caminho mais curto em grafos não ponderados.

```
from collections import deque

def bfs(grafo, inicio):
    visitados = set()
    fila = deque([inicio])
    while fila:
        vertice = fila.popleft()
        if vertice not in visitados:
            print(vertice)
            visitados.add(vertice)
            fila.extend([viz for viz in grafo[vertice] if viz not in
visitados])
```

Busca em profundidade (DFS)

A busca em profundidade (Depth-First Search) explora o grafo indo o mais fundo possível em cada ramo antes de retroceder. Utiliza uma pilha (stack) ou recursão para controlar a ordem de visitação. É útil para detectar ciclos, componentes conexos e para percorrer todos os caminhos possíveis em um grafo.

```python
def dfs(grafo, inicio, visitados=None):
    if visitados is None:
        visitados = set()
    print(inicio)
    visitados.add(inicio)
    for vizinho in grafo[inicio]:
        if vizinho not in visitados:
            dfs(grafo, vizinho, visitados)
```

Grafo com pesos

```python
grafo_pesos = {
    'A': [('B', 2), ('C', 5)],
    'B': [('A', 2), ('D', 1)],
    'C': [('A', 5), ('D', 3)],
    'D': [('B', 1), ('C', 3)]
}
```

Usando NetworkX (biblioteca para grafos)

```python
import networkx as nx

G = nx.Graph()
G.add_edge('A', 'B')
G.add_edge('A', 'C')
G.add_edge('B', 'D')

print(list(G.nodes))      # ['A', 'B', 'C', 'D']
print(list(G.edges))      # [('A', 'B'), ('A', 'C'), ('B', 'D')]
```

Dica: Use dicionários para grafos simples e a biblioteca `networkx` para grafos mais complexos e algoritmos prontos!

---

6 Linear Scan

Buscar elemento em uma lista

```python
nums = [3, 7, 2, 9, 5]
print(9 in nums)         # True
print(nums.index(9))     # 3 (índice do elemento 9)
```

Encontrar o maior e o menor elemento

```
nums = [3, 7, 2, 9, 5]
print(max(nums))         # 9
print(min(nums))         # 2
```

Somar todos os elementos

```
nums = [1, 2, 3, 4]
print(sum(nums))         # 10
```

Contar ocorrências de um valor

```
nums = [1, 2, 2, 3, 2, 4]
print(nums.count(2))     # 3
```

Filtrar elementos (usando list comprehensions ou filter)

```
nums = [1, 2, 3, 4, 5]
pares = [n for n in nums if n % 2 == 0]  # [2, 4]

# Ou usando filter
pares = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]
```

Encontrar o índice de um elemento

```
letras = ['a', 'b', 'c', 'd']
print(letras.index('c'))    # 2
```

Verificar se todos/algum elemento satisfaz uma condição

```
nums = [2, 4, 6]
print(all(n % 2 == 0 for n in nums))   # True (todos pares)
print(any(n > 5 for n in nums))        # True (algum maior que 5)
```

Iterar com índice (enumerate)

```
frutas = ['maçã', 'banana', 'uva']
for i, fruta in enumerate(frutas):
    print(i, fruta)
```

Dica:
Use as funções e métodos nativos do Python para tornar seu código mais limpo, eficiente e legível!

---

7 Algoritmos Gulosos

Ordenar elementos (para pegar maiores/menores rapidamente)

```
nums = [5, 2, 8, 1, 9]
nums.sort()           # Ordena em ordem crescente
nums.sort(reverse=True)  # Ordena em ordem decrescente

# Obter os 3 maiores elementos
maiores = sorted(nums, reverse=True)[:3]  # [9, 8, 5]
```

Obter maiores/menores elementos rapidamente

```
nums = [5, 2, 8, 1, 9]
print(max(nums))      # 9 (maior)
print(min(nums))      # 1 (menor)

# Usando sorted para pegar os k menores/maiores
menores = sorted(nums)[:2]      # [1, 2]
maiores = sorted(nums)[-2:]     # [8, 9]
```

Soma acumulada (para decisões rápidas)

```
nums = [1, 2, 3, 4]
print(sum(nums))      # 10
```

Selecionar itens por critério (list comprehensions)

```
# Selecionar itens menores que 5
nums = [1, 4, 6, 8, 3]
selecionados = [n for n in nums if n < 5]  # [1, 4, 3]
```

Emparelhar itens de listas ordenadas (ex: problema do troco)

```
moedas = [1, 5, 10, 25]
valor = 37
moedas.sort(reverse=True)  # Guloso: usar as maiores moedas primeiro
```

```
resultado = []
for moeda in moedas:
    while valor >= moeda:
        valor -= moeda
        resultado.append(moeda)
print(resultado)  # [25, 10, 1, 1]
```

Verificar se todos/algum elemento satisfaz uma condição (Greedy)

```
nums = [2, 4, 6]

# all(): retorna True se TODOS os elementos satisfazem a condição
print(all(n % 2 == 0 for n in nums))   # True (todos são pares)

# any(): retorna True se ALGUM elemento satisfaz a condição
print(any(n > 5 for n in nums))        # True (algum é maior que 5)
```

Iterar com índice usando enumerate

```
frutas = ['maçã', 'banana', 'uva']

for i, fruta in enumerate(frutas):
    print(i, fruta)
# Saída:
# 0 maçã
# 1 banana
# 2 uva
```

Funções úteis para algoritmos gulosos

- `max()`, `min()`: Encontrar maior/menor rapidamente.
- `sorted()`: Ordenar elementos.
- `sum()`: Soma total.
- `any()`, `all()`: Verificar condições em coleções.
- `enumerate()`: Iterar com índice.

Dica:
Para muitos problemas gulosos clássicos (troco, seleção de intervalos, agendamento), combine ordenação, filtros e funções nativas do Python para soluções rápidas e eficientes!

---

8 Programação Dinâmica

Técnicas de Programação Dinâmica

1. Memoização (Top-Down)

- Armazena os resultados de subproblemas já resolvidos para evitar cálculos repetidos.
- Implementado geralmente com recursão e um dicionário ou cache.

2. Tabulação (Bottom-Up)

- Resolve os subproblemas menores primeiro e usa os resultados para construir a solução do problema maior.
- Implementado iterativamente com tabelas (listas ou matrizes).

Exemplo 1: Fibonacci (Memoização)

```
def fibonacci(n, memo={}):
    if n <= 1:
        return n
    if n not in memo:
        memo[n] = fibonacci(n-1, memo) + fibonacci(n-2, memo)
    return memo[n]

print(fibonacci(10))  # 55
```

Exemplo 2: Fibonacci (Tabulação)

```
def fibonacci(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]

print(fibonacci(10))  # 55
```

Exemplo 3: Problema da Mochila 0/1 (Tabulação)

```
def mochila(pesos, valores, capacidade):
    n = len(pesos)
    dp = [[0] * (capacidade + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacidade + 1):
            if pesos[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w], valores[i - 1] + dp[i - 1][w
- pesos[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]
```

```
    return dp[n][capacidade]

pesos = [1, 2, 3]
valores = [10, 15, 40]
capacidade = 5
print(mochila(pesos, valores, capacidade))  # 55
```

Exemplo 4: Caminhos em uma Grade (Tabulação)

```
def caminhos_na_grade(m, n):
    dp = [[1] * n for _ in range(m)]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i - 1][j] + dp[i][j - 1]
    return dp[m - 1][n - 1]

print(caminhos_na_grade(3, 3))  # 6
```

Exemplo 5: Soma Máxima de Subarray (Kadane's Algorithm)

```
def soma_maxima_subarray(nums):
    max_atual = max_global = nums[0]
    for num in nums[1:]:
        max_atual = max(num, max_atual + num)
        max_global = max(max_global, max_atual)
    return max_global

nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(soma_maxima_subarray(nums))  # 6
```

Exemplo 6: Sebset Sum Problem

```
def subset_sum(nums, target):
    n = len(nums)
    dp = [[False] * (target + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = True
    for i in range(1, n + 1):
        for t in range(1, target + 1):
            if nums[i-1] <= t:
                dp[i][t] = dp[i-1][t] or dp[i-1][t - nums[i-1]]
            else:
                dp[i][t] = dp[i-1][t]
    return dp[n][target]
```

```
print(subset_sum([3, 34, 4, 12, 5, 2], 9))  # Output: True
```

Exemplo 7: Longest Common Subsequence (LCS)

```
def lcs(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[n][m]

print(lcs("ABCBDAB", "BDCAB"))  # Output: 4
```

Exemplo 8: Minimum Edit Distance (Levenshtein Distance)

```
def edit_distance(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0] * (m+1) for _ in range(n+1)]
    for i in range(n+1):
        for j in range(m+1):
            if i == 0:
                dp[i][j] = j
            elif j == 0:
                dp[i][j] = i
            elif s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[n][m]

print(edit_distance("kitten", "sitting"))  # Output: 3
```

Funções Úteis para Programação Dinâmica

- `max()`: Para encontrar o valor máximo em subproblemas.
- `min()`: Para encontrar o valor mínimo em subproblemas.
- `sum()`: Para somar valores acumulados.
- `range()`: Para iterar em tabulação.
- **Dicionários**: Para memoização.

Dica:
Use memoização para problemas recursivos e tabulação para problemas iterativos. Identifique

subproblemas repetidos para aplicar programação dinâmica de forma eficiente!

---

9 Busca Binária

Requisitos

- A coleção deve estar **ordenada**.

Implementação Básica de Busca Binária

```
def busca_binaria(arr, alvo):
    inicio, fim = 0, len(arr) - 1
    while inicio <= fim:
        meio = (inicio + fim) // 2
        if arr[meio] == alvo:
            return meio  # Retorna o índice do elemento
        elif arr[meio] < alvo:
            inicio = meio + 1
        else:
            fim = meio - 1
    return -1  # Retorna -1 se o elemento não for encontrado

# Exemplo de uso
nums = [1, 3, 5, 7, 9]
print(busca_binaria(nums, 5))  # 2
print(busca_binaria(nums, 4))  # -1
```

Usando `bisect` (Biblioteca Padrão)

O módulo `bisect` fornece funções para trabalhar com busca binária em listas ordenadas.

Inserir um elemento mantendo a ordem

```
import bisect

nums = [1, 3, 5, 7, 9]
bisect.insort(nums, 6)  # Insere 6 na posição correta
print(nums)  # [1, 3, 5, 6, 7, 9]
```

Encontrar a posição de inserção

```
import bisect

nums = [1, 3, 5, 7, 9]
pos = bisect.bisect(nums, 6)  # Retorna a posição onde 6 seria inserido
print(pos)  # 3
```

Busca Binária Recursiva

```
def busca_binaria_recursiva(arr, alvo, inicio, fim):
    if inicio > fim:
        return -1
    meio = (inicio + fim) // 2
    if arr[meio] == alvo:
        return meio
    elif arr[meio] < alvo:
        return busca_binaria_recursiva(arr, alvo, meio + 1, fim)
    else:
        return busca_binaria_recursiva(arr, alvo, inicio, meio - 1)

# Exemplo de uso
nums = [1, 3, 5, 7, 9]
print(busca_binaria_recursiva(nums, 5, 0, len(nums) - 1))  # 2
print(busca_binaria_recursiva(nums, 4, 0, len(nums) - 1))  # -1
```

Encontrar o Primeiro ou Último Índice de um Elemento

```
import bisect

nums = [1, 3, 3, 3, 5, 7, 9]

# Primeiro índice de 3
primeiro = bisect.bisect_left(nums, 3)
print(primeiro)  # 1

# Último índice de 3
ultimo = bisect.bisect_right(nums, 3) - 1
print(ultimo)  # 3
```

Verificar Existência de um Elemento

```
import bisect

nums = [1, 3, 5, 7, 9]
alvo = 5

pos = bisect.bisect_left(nums, alvo)
if pos < len(nums) and nums[pos] == alvo:
    print("Elemento encontrado!")
else:
    print("Elemento não encontrado!")
```

Aplicações Comuns da Busca Binária

1. **Encontrar um elemento em uma lista ordenada.**
2. **Determinar a posição de inserção de um elemento.**
3. **Resolver problemas de otimização (ex: encontrar o menor valor que satisfaz uma condição).**

Dica:
Use a busca binária sempre que precisar trabalhar com coleções ordenadas para melhorar a eficiência do seu algoritmo!

---

10 Permutação e Contagem de Ciclos

Gerar Permutações com `itertools.permutations`

```
from itertools import permutations

# Gerar todas as permutações de uma lista
nums = [1, 2, 3]
perm = list(permutations(nums))
print(perm)
# Saída: [(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2,
1)]
```

Contar Permutações

```
from math import factorial

# Número de permutações de n elementos
n = 3
print(factorial(n))  # 6 (3! = 3 × 2 × 1)
```

Permutação com Repetição

```
from itertools import product

# Gerar permutações com repetição
nums = [1, 2]
perm_repet = list(product(nums, repeat=3))
print(perm_repet)
# Saída: [(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2), (2, 1, 1), (2, 1,
2), (2, 2, 1), (2, 2, 2)]
```

Encontrar Ciclos em Permutações

```python
def contar_ciclos(perm):
    n = len(perm)
    visitado = [False] * n
    ciclos = 0

    for i in range(n):
        if not visitado[i]:
            ciclos += 1
            j = i
            while not visitado[j]:
                visitado[j] = True
                j = perm[j]
    return ciclos

# Exemplo de uso
perm = [2, 0, 1]  # Representa a permutação [1, 2, 0]
print(contar_ciclos(perm))  # 1 (um ciclo completo)
```

Reorganização de Linhas e Colunas (Problema de Ciclos)

Exemplo: Reorganizar uma Matriz

```python
def reorganizar_matriz(matriz, perm_linhas, perm_colunas):
    n = len(matriz)
    nova_matriz = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            nova_matriz[i][j] = matriz[perm_linhas[i]][perm_colunas[j]]
    return nova_matriz

# Exemplo de uso
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
perm_linhas = [2, 0, 1]
perm_colunas = [1, 2, 0]
nova_matriz = reorganizar_matriz(matriz, perm_linhas, perm_colunas)
print(nova_matriz)
# Saída:
# [[8, 9, 7],
#  [2, 3, 1],
#  [5, 6, 4]]
```

Aplicações Comuns

1. **Problemas de ordenação e combinação.**
2. **Análise de ciclos em grafos ou permutações.**

3. **Reorganização de dados em matrizes.**

Dica:
Use `itertools` para gerar permutações e `factorial` para calcular o número total de permutações. Para análise de ciclos, implemente algoritmos personalizados para maior controle!

---

11 Algoritmo de Kruskal

Estrutura do Algoritmo

1. Ordenar todas as arestas pelo peso.
2. Adicionar as arestas de menor peso ao MST, desde que não formem ciclos.
3. Repetir até que todos os vértices estejam conectados.

Implementação Básica com Funções Nativas

```
# Função para encontrar o representante de um conjunto (usando Union-Find)
def find(pai, vertice):
    if pai[vertice] != vertice:
        pai[vertice] = find(pai, pai[vertice])  # Compressão de caminho
    return pai[vertice]

# Função para unir dois conjuntos
def union(pai, rank, vertice1, vertice2):
    raiz1 = find(pai, vertice1)
    raiz2 = find(pai, vertice2)
    if raiz1 != raiz2:
        if rank[raiz1] > rank[raiz2]:
            pai[raiz2] = raiz1
        elif rank[raiz1] < rank[raiz2]:
            pai[raiz1] = raiz2
        else:
            pai[raiz2] = raiz1
            rank[raiz1] += 1

# Algoritmo de Kruskal
def kruskal(vertices, arestas):
    # Ordenar as arestas pelo peso
    arestas.sort(key=lambda x: x[2])
    pai = {v: v for v in vertices}
    rank = {v: 0 for v in vertices}
    mst = []

    for vertice1, vertice2, peso in arestas:
        if find(pai, vertice1) != find(pai, vertice2):
            union(pai, rank, vertice1, vertice2)
            mst.append((vertice1, vertice2, peso))

    return mst

# Exemplo de uso
```

```
vertices = ['A', 'B', 'C', 'D', 'E']
arestas = [
    ('A', 'B', 1),
    ('A', 'C', 5),
    ('B', 'C', 4),
    ('B', 'D', 2),
    ('C', 'D', 6),
    ('D', 'E', 3)
]

mst = kruskal(vertices, arestas)
print(mst)
# Saída: [('A', 'B', 1), ('B', 'D', 2), ('D', 'E', 3), ('B', 'C', 4)]
```

Explicação do Código

1. **Estruturas Union-Find**:
   - `find`: Encontra o representante de um conjunto (com compressão de caminho para eficiência).
   - `union`: Une dois conjuntos, usando a técnica de união por rank.
2. **Ordenação**:
   - As arestas são ordenadas pelo peso usando `sort` com `key=lambda x: x[2]`.
3. **Construção da MST**:
   - A cada iteração, verifica-se se a aresta forma um ciclo usando `find`.
   - Se não formar ciclo, a aresta é adicionada à MST.

Aplicações do Algoritmo de Kruskal

1. **Redes de computadores**: Construir redes de menor custo.
2. **Planejamento de infraestrutura**: Estradas, linhas de energia, etc.
3. **Problemas de conectividade**: Encontrar conexões mínimas em grafos.

Dica:
Use estruturas como dicionários e listas para implementar o algoritmo de Kruskal de forma eficiente com as funções nativas do Python!

---

12 Algoritmo de Dijkstra

Estrutura do Algoritmo (Dijkstra)

1. Inicializar a distância de todos os vértices como infinita, exceto o vértice de origem (distância 0).
2. Usar uma fila de prioridade para explorar os vértices com a menor distância conhecida.
3. Atualizar as distâncias dos vizinhos do vértice atual.
4. Repetir até que todos os vértices tenham sido processados.

Implementação Básica com Funções Nativas (Dijkstra)

```python
import heapq

def dijkstra(grafo, origem):
    # Inicializar distâncias e fila de prioridade
    distancias = {v: float('inf') for v in grafo}
    distancias[origem] = 0
    fila = [(0, origem)]  # (distância, vértice)

    while fila:
        distancia_atual, vertice_atual = heapq.heappop(fila)

        # Verificar os vizinhos do vértice atual
        for vizinho, peso in grafo[vertice_atual]:
            distancia = distancia_atual + peso

            # Atualizar a distância se for menor
            if distancia < distancias[vizinho]:
                distancias[vizinho] = distancia
                heapq.heappush(fila, (distancia, vizinho))

    return distancias

# Exemplo de uso
grafo = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 5)],
    'C': [('A', 4), ('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)]
}

distancias = dijkstra(grafo, 'A')
print(distancias)
# Saída: {'A': 0, 'B': 1, 'C': 3, 'D': 4}
```

Explicação do Código (Dijkstra)

1. **Inicialização**:
   - Todas as distâncias começam como `infinito`, exceto o vértice de origem (distância 0).
   - A fila de prioridade (`heapq`) é usada para processar os vértices com a menor distância conhecida.
2. **Atualização de Distâncias**:
   - Para cada vizinho do vértice atual, calcula-se a nova distância.
   - Se a nova distância for menor que a distância armazenada, ela é atualizada e o vizinho é adicionado à fila.
3. **Fila de Prioridade**:
   - A fila garante que os vértices sejam processados na ordem de menor distância.

Representação do Grafo

- O grafo é representado como um dicionário, onde as chaves são os vértices e os valores são listas de tuplas `(vizinho, peso)`.

Exemplo:

```
grafo = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 5)],
    'C': [('A', 4), ('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)]
}
```

Aplicações do Algoritmo de Dijkstra

1. **Redes de computadores**: Encontrar o caminho mais curto entre roteadores.
2. **Mapas e navegação**: Calcular rotas mais curtas em sistemas de GPS.
3. **Problemas de otimização**: Resolver problemas de menor custo em grafos ponderados.

Dica:
Use o módulo `heapq` para implementar a fila de prioridade de forma eficiente. Certifique-se de que os pesos do grafo sejam não negativos para usar o algoritmo de Dijkstra corretamente!

---

13 Bitmask e Busca Exaustiva

Representação de Subconjuntos com Bitmask

- Cada bit de um número inteiro representa a inclusão (`1`) ou exclusão (`0`) de um elemento em um subconjunto.
- Por exemplo, para um conjunto `[A, B, C]`:
  - `000` → Subconjunto vazio.
  - `101` → Subconjunto `{A, C}`.

Gerar Todos os Subconjuntos

```
def gerar_subconjuntos(nums):
    n = len(nums)
    subconjuntos = []
    for bitmask in range(1 << n):  # Gera todos os números de 0 a 2^n - 1
        subconjunto = [nums[i] for i in range(n) if bitmask & (1 << i)]
        subconjuntos.append(subconjunto)
    return subconjuntos

# Exemplo de uso
nums = [1, 2, 3]
print(gerar_subconjuntos(nums))
# Saída: [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
```

Contar Subconjuntos que Satisfazem uma Condição

```
def contar_subconjuntos(nums, alvo):
    n = len(nums)
    contador = 0
    for bitmask in range(1 << n):
        soma = sum(nums[i] for i in range(n) if bitmask & (1 << i))
        if soma == alvo:
            contador += 1
    return contador

# Exemplo de uso
nums = [1, 2, 3]
alvo = 3
print(contar_subconjuntos(nums, alvo))  # 2 ([3], [1, 2])
```

Busca Exaustiva com Bitmask

Problema: Subconjunto com Soma Máxima Menor que um Limite

```
def soma_maxima(nums, limite):
    n = len(nums)
    melhor_soma = 0
    for bitmask in range(1 << n):
        soma = sum(nums[i] for i in range(n) if bitmask & (1 << i))
        if soma <= limite:
            melhor_soma = max(melhor_soma, soma)
    return melhor_soma

# Exemplo de uso
nums = [3, 5, 8]
limite = 10
print(soma_maxima(nums, limite))  # 8 ([3, 5])
```

Operações Úteis com Bitmask

Verificar se um bit está ativo

```
bitmask = 5  # 101 em binário
pos = 1
print(bool(bitmask & (1 << pos)))  # True (o bit na posição 1 está ativo)
```

Ativar um bit

```
bitmask = 5  # 101 em binário
pos = 2
bitmask |= (1 << pos)
print(bin(bitmask))  # 0b111
```

Desativar um bit

```
bitmask = 7  # 111 em binário
pos = 1
bitmask &= ~(1 << pos)
print(bin(bitmask))  # 0b101
```

Alternar um bit

```
bitmask = 5  # 101 em binário
pos = 1
bitmask ^= (1 << pos)
print(bin(bitmask))  # 0b111
```

Aplicações Comuns de Bitmask

1. **Problemas de subconjuntos**: Encontrar subconjuntos que satisfazem uma condição.
2. **Problemas de estados**: Representar estados em problemas de programação dinâmica.
3. **Problemas de combinação**: Resolver problemas de busca exaustiva de forma eficiente.

Dica:
Use operações bit a bit para manipular subconjuntos de forma eficiente. A técnica de bitmask é especialmente útil para conjuntos pequenos (até 20 elementos) devido à explosão combinatória!

---

14 Problemas Clássicos: Caixeiro Viajante e Torre de Hanói

Problema do Caixeiro Viajante (Traveling Salesman Problem - TSP)

Solução com Busca Exaustiva (Bitmask + Programação Dinâmica)

```
def tsp(grafo, n):
    # Inicializar a tabela de DP
    dp = [[float('inf')] * (1 << n) for _ in range(n)]
    dp[0][1] = 0  # Começa na cidade 0 com o bitmask 1 (apenas cidade 0
visitada)

    # Preencher a tabela de DP
    for mask in range(1 << n):
```

```
        for u in range(n):
            if mask & (1 << u):  # Se a cidade u está no subconjunto
representado por mask
                for v in range(n):
                    if not mask & (1 << v):  # Se a cidade v não está no
subconjunto
                        dp[v][mask | (1 << v)] = min(dp[v][mask | (1 <<
v)], dp[u][mask] + grafo[u][v])

    # Retornar o menor custo para visitar todas as cidades e voltar à
inicial
    return min(dp[i][(1 << n) - 1] + grafo[i][0] for i in range(n))

# Exemplo de uso
grafo = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(grafo)
print(tsp(grafo, n))  # Saída: 80
```

Explicação do Código - Caixeiro Viajante

1. **Bitmask**:
   - Cada bit do número representa se uma cidade foi visitada (`1`) ou não (`0`).
2. **Tabela de DP**:
   - `dp[u][mask]` armazena o menor custo para visitar o subconjunto de cidades representado por `mask`, terminando na cidade `u`.
3. **Transição**:
   - Para cada cidade `u` no subconjunto atual, tenta-se adicionar uma nova cidade `v` ao subconjunto e atualiza-se o custo mínimo.

Problema da Torre de Hanói

Solução Recursiva

```
def torre_de_hanoi(n, origem, destino, auxiliar):
    if n == 1:
        print(f"Mova o disco 1 de {origem} para {destino}")
        return
    torre_de_hanoi(n - 1, origem, auxiliar, destino)
    print(f"Mova o disco {n} de {origem} para {destino}")
    torre_de_hanoi(n - 1, auxiliar, destino, origem)

# Exemplo de uso
n = 3  # Número de discos
torre_de_hanoi(n, 'A', 'C', 'B')
```

Saída para `n = 3`

```
Mova o disco 1 de A para C
Mova o disco 2 de A para B
Mova o disco 1 de C para B
Mova o disco 3 de A para C
Mova o disco 1 de B para A
Mova o disco 2 de B para C
Mova o disco 1 de A para C
```

Explicação do Código - Torre de Hanoi

1. **Caso Base**:
   - Se houver apenas 1 disco, mova-o diretamente do pino de origem para o pino de destino.
2. **Passo Recursivo**:
   - Mova os $n-1$ discos do pino de origem para o pino auxiliar.
   - Mova o disco maior do pino de origem para o pino de destino.
   - Mova os $n-1$ discos do pino auxiliar para o pino de destino.

Aplicações dos Problemas

Caixeiro Viajante

1. **Logística e transporte**: Planejar rotas de entrega.
2. **Circuitos eletrônicos**: Minimizar o comprimento de conexões.

Torre de Hanói

1. **Análise de recursão**: Problema clássico para entender recursão.
2. **Sistemas de armazenamento**: Simular movimentação de dados entre discos.

Dica:
Para o problema do Caixeiro Viajante, use bitmask e programação dinâmica para resolver instâncias pequenas de forma eficiente. Para a Torre de Hanói, a solução recursiva é direta e elegante!